JAVA RIASSUNTO PROGRAMMA

1. **INTRODUZIONE**
2. **LINGUAGGIO JAVA**
3. **PROGRAMMAZIONE AD OGGETTI IN JAVA**
    a. Ereditarietà, interfacce e packages
    b. Polimorfismo sui dati
    c. Typecasting
    d. Array
4. **MANIPOLAZIONE DEGLI OGGETTI**
    a. Costanti
    b. Campi e metodi di classe
    c. Confronto tra oggetti
    d. Incapsulazione
    e. this
    f. Passaggio dei parametri
    g. Overloading
    h. Classi e metodi astratti
5. **COMPLESSITA'**
    a. Costo computazionale
6. **RICORSIONE**
    a. Numeri di Fibonacci
    b. Elevamento a potenza
    c. Ricerca binaria (con versione iterativa)
    d. Quicksort (con versione iterativa)
    e. Torri di Hanoi
    f. Bubblesort
7. **ECCEZIONI**
    a. Protezione dalle eccezioni
    b. Dichiarazione di metodi che sollevano eccezioni
    c. Creazione e lancio di eccezioni
8. **FLUSSI DI INPUT E OUTPUT**
    a. Flussi di byte
    b. Flussi di caratteri
    c. Chiusura dei flussi
9. **MULTITHREADING**
    a. Creazione e uso di thread
    b. Sincronizzazione
10. **PROGRAMMAZIONE DI RETE**
    a. Connessioni tramite socket
    b. SocketStream
    c. Cosa fare per esame

INTRODUZIONE

I motivi principali per cui l'Object Oriented Programming è efficace sono:

1. l'aumento della produttività
2. facilitare il programmatore
3. velocizzare lo sviluppo
4. ridurre gli errori

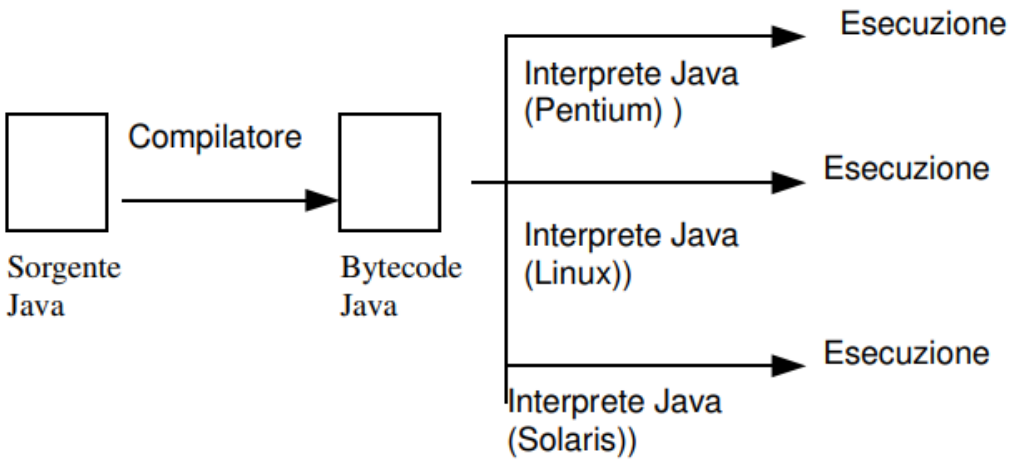
Nel caso di un linguaggio come C, la programmazione è strutturata, vale a dire un problema viene diviso in sottoproblemi più semplici, a loro volta divisi in sottoprogrammi, come funzioni e procedure (metodo di top-down)

Nel caso, invece, dell'OOP, si costruisce un'applicazione montando oggetti più semplici; questi oggetti vengono progettati tramite classi, costruiti ed infine usati.
Per progettare un oggetto si devono indicare le sue proprietà (attributi o campi) e le sue funzionalità(metodi(*o funzioni*)).
I tre pilastri dell'OOP sono:

1. ereditarietà (è possibile riutilizzare un progetto già esistente e modificarlo per le proprie esigenze)
2. incapsulazione(riuso del codice, astrazione, confinamento errori)
3. polimorfismo (ogni oggetto si comporta a seconda del contesto e del valore)

1 LINGUAGGIO JAVA

L'ambiente Java comprende sia un compilatore che un interprete (Java Runtime Environment, JRE).



Attraverso il compilatore, la sorgente Java viene convertita in bytecode, pseudo istruzioni in un linguaggio inventato indipendenti dalla piattaforma.
Gli interpreti traducono il bytecode in un eseguibile specifico a seconda del sistema operativo in cui si trova il bytecode (Windows, Linux, Mac).
Questo permette una compensazione tra velocità e portabilità.

2 PROGRAMMAZIONE AD OGGETTI IN JAVA

La classe è il modello generico di un insieme di oggetti con caratteristiche simili.
L'oggetto è la particolare istanza di un elemento di una classe.

Gli attributi (proprietà e campi) descrivono lo stato, l'aspetto e altre qualità dell'oggetto; sono rappresentati dai campi dell'oggetto. Esistono anche campi di classe.

Il comportamento di una classe indica come opera un oggetto di una classe; è definito dai metodi delle classi, funzioni definite all'interno della classe che operano sugli oggetti della classe e ne manipolano le proprietà. Non esistono funzioni definite al di fuori di una classe.

Come viene creato un oggetto?
Motocicletta m; *//dichiarazione (creo variabile m di tipo Motocicletta); m occupa 4 byte*
m = new Motocicletta(); *//istanziazione (creo la nuova motocicletta)*
m.marca = "Guzzi 1000"; *//inizializzazione*
m.colore = "rossa"; *//inizializzazione (inserisco le proprietà alla variabile vuota m)*
m.mostra(); *//prende il metodo "mostra", e i valori restituiti vengono dati ad m*

Come viene creato un oggetto con un costruttore?
Cosa fa un costruttore?
1) istanzia l'oggetto e ne restituisce l'indirizzo
2) quello che il programmatore vuole; di solito inizializza l'oggetto (assegna alle proprietà il valore iniziale, di solito prendendolo da parametri)

public Motocicletta(String marca, String colore, boolean a) { <- *costruttore*
this.marca= marca; this.colore = colore; accesa = a; //*assegno il valore del parametro alla proprietà dell'oggetto*}

public static void main (String args[]) {
Motocicletta m=new Motocicletta("Guzzi 1000","rossa",false); //*L'oggetto è dichiarato e istanziato nella stessa riga; viene inizializzato attraverso i parametri*}

Come si comporta la memoria?

1) STACK: Vengono rappresentati gli spazi in cui i metodi vengono eseguiti, con al loro interno i parametri e le variabili locali. E' sempre presente il metodo *main* con il parametro *String args[]* e le sue variabili (in genere, i nomi degli oggetti inizializzati). Gli spazi dei metodi non statici rimangono fino a quando questi hanno smesso la loro esecuzione.
2) HEAP: Vengono rappresentati gli spazi indicati dagli oggetti da costruire, con all'interno le sue proprietà.
3) STATIC: Vengono inserite le variabili statiche, che possono essere usate in quanto definite.

Quando un oggetto viene dichiarato, la variabile, che dà il nome all'oggetto, è un numero in base 16 che indica un indirizzo di memoria, il quale indica dove si trova l'oggetto vero e proprio; in questo momento, l'indirizzo è vuoto. La variabile è un oggetto per riferimento.
L'oggetto viene creato con l'istanziazione, attraverso i costruttori; viene allocato uno spazio di memoria nell'heap per l'oggetto e le sue proprietà. Il nome dell'oggetto contiene la posizione in memoria di questo spazio di memoria.
Quando l'oggetto viene inizializzato, le proprietà associate all'oggetto vengono trasferite all'interno del suo spazio nello heap, grazie al nome dell'oggetto che le marca come sue e le guida con l'indirizzo. Oggetti inutilizzati vengono deallocati automaticamente tramite un meccanismo di garbage collection. Per accedere ai campi di un oggetto e per richiamare un metodo si usa la notazione punto.
L'oggetto finale è formato dal suo indirizzo di memoria e dallo spazio assegnato all'indirizzo, con al suo interno le sue proprietà.

Il compito del compilatore e dell'interprete

1) COMPILATORE: Esegue controlli statici, basandosi sulle dichiarazioni delle variabili, e controlla se una variabile sia inizializzata o no.
2) INTERPRETE: esegue controlli dinamici, basandosi sui valori delle variabili:
    - si utilizza un oggetto che non è stato costruito
    - si accede alla posizione di un array inesistente
    - si calcola il valore indicato da una stringa che non rappresenta un numero
    - si considera il valore di un oggetto appartenente ad una classe con cui non è compatibile

Ereditarietà, interfacce e packages

E' possibile definire una sottoclasse, discendente da una sovraclasse, che ne eredita campi e metodi (*extends*) (gli elementi privati vengono ereditati, ma non sono direttamente accessibili); questa è l'ereditarietà.
Java usa l'ereditarietà singola, ovvero ogni classe può avere una sola sovraclasse; ciò serve per mantenere semplice il linguaggio, e quindi più sicuro il compilatore e l'interprete.
Se ci fosse l'ereditarietà multipla, la classe potrebbe estendere 2 sovraclassi S1 e S2; se S1 ed S2 contenessero campi omonimi, con una diversa occupazione di memoria, servirebbe uno strumento per dire quale delle 2 usare nella sottoclasse e sarebbero poi necessari controlli a tempo di esecuzione per verificare che il programmatore continui ad usare il campo che ha scelto.

Due approcci:

1) *bottom-up*: da classi più semplici si costruiscono più complicate
2) *top-down*: le classi vengono organizzate in una gerarchia a forma di albero, che parte da sovraclassi che rappresentano proprietà e metodi comuni, fino a più sottoclassi con elementi più specifici.

Le interfacce sono collezioni di dichiarazioni di metodi che possono essere aggiunti alle classi e consentono di avere comportamenti analoghi in rami diversi della gerarchia delle classi. (*interface / implements*). Una classe può estendere solo un'altra classe, ma può implementare tutte le interfacce che vuole. Un'interfaccia è una collezione di:

1. intestazioni di metodi di istanza astratti;
2. costanti statiche;
3. metodi default implementati (lavorano con i propri parametri e sono ridefinibili nelle classi che implementano l'interfaccia)
4. metodi statici implementati (non ridefinibili nelle classi che implementano l'interfaccia)

Un package è una collezione di classi ed interfacce; consente di rendere disponibili vari gruppi prestabiliti di classi quando necessario e permette di eliminare conflitti riguardo i nomi delle classi. Per default sono disponibili solo le classi contenute nel package java.lang; altre classi devono essere riferite esplicitamente o importate. (*import*)

Polimorfismo sui dati
Un oggetto di una certa classe può assumere valori di oggetti di una qualsiasi sottoclasse. Si può dichiarare un oggetto di tipo interfaccia (invece che di tipo classe), allora l'oggetto può assumere come valore quello di una qualsiasi classe che implementa l'interfaccia.

Attraverso l'astrazione (la possibilità di inserire in una sopraclasse le caratteristiche comuni delle classi di un'applicazione) ho la possibilità di definire strutture dati polimorfe (cioè che contengono oggetti di classi diverse //array).
ES //Lavoratore è la sopraclasse
Lavoratore l3= new Operaio(); l3.stipendio();

1. il metodo stipendio è definito nella classe Lavoratore.
2. dato che il valore di i3 è un operaio, viene eseguito il metodo stipendio della classe operaio.

l3 è un lavoratore oppure un operaio?

1. secondo il compilatore, I3 è un Lavoratore (il compilatore considera gli oggetti secondo la loro dichiarazione e controlla se I3 sia utilizzato al fatto di essere un Lavoratore //**Early binding**)
2. secondo l'interprete, I3 è un Operaio (in fase di esecuzione, I3 viene trattato secondo il valore che assume, ovvero come Operaio // **Late binding**)

Java, quando possibile, usa sempre il late binding, ad eccezione di metodi di classe statici, metodi di classi final (non esistono le sottoclassi) e metodi final (nelle sottoclassi non è possibile ridefinire il metodo)..
Il compilatore effettua comunque un controllo statico (cioè basato sulle dichiarazioni) sulla chiamata del metodo, verificando che il metodo sia definito nella (o ereditato dalla) classe a cui abbiamo dichiarato che l'oggetto appartiene.

Typecasting
Nel typecasting di oggetti il programmatore indica (al compilatore) la classe del valore che l'oggetto contiene. In questo modo nei suoi controlli statici il compilatore non tiene conto della classe a cui si è dichiarato che l'oggetto appartiene ma della classe su cui abbiamo fatto il typecasting.
Se durante l'esecuzione il valore dell'oggetto non risulta compatibile con la classe su cui si è fatto il typecasting ci sarà un errore in esecuzione.

Array
Un array è un insieme di elementi dello stesso tipo che possono essere inseriti senza limitazioni, Essendo un oggetto, l'array deve essere dichiarato ed istanziato con una chiamata di costruttore.
La creazione di un array viene effettuata in due modi:

1) si usa il metodo new (la dimensione dell'array viene indicata esplicitamente)
2) si elenca il contenuto dell'array al momento della dichiarazione (la dimensione è implicita)

Il campo length, comune a tutti gli oggetti array, contiene il numero di componenti dell'array.

3 MANIPOLAZIONE DEGLI OGGETTI

Come assegnare un valore ad un oggetto?

1) chiamata di costruttore
2) assegnare *null* ad un oggetto
3) assegnare un valore costante non nullo
4) assegnare il valore restituito da un metodo (che deve essere un oggetto compatibile)
5) assegnare il valore di un altro oggetto (ovvero la stessa locazione in memoria)

Costanti

Una costante può essere definita usando la parola **final**; questa costante non può essere più modificata. Viene usata per proprietà o variabili (classi e metodi).
Non possono essere definite sottoclassi di una classe final, e un metodo final non può essere ridefinito nelle sottoclassi.
Metodi final o metodi di classe final non sono trattati con il polimorfismo, ovvero le chiamate sono risolte a tempo di compilazione.

Campi e metodi di classe

I campi di classe sono preceduti dalla parola **static**; rappresentano proprietà che non sono specifiche dei singoli oggetti della classe, ma si riferiscono alla classe stessa; oppure, rappresentano proprietà globali rispetto agli oggetti della classe. Sono proprietà uniche di una sola classe, che nessun'altra può avere.
Le proprietà statiche sono utilizzabili anche in assenza di oggetti nella classe, e sono utilizzabili per tutta la durata dell'applicazione.
Sono accessibili con la sintassi:
NomeClasse.nomeproprietà (o anche così: oggetto.nomeproprietà)

I metodi di classe sono preceduti da **static** e sono utilizzabili anche in assenza di oggetti nella classe.
Sono richiamabili con la sintassi:
NomeClasse.nomemetodo(); (o anche così: oggetto.nomemetodo();)
La parola chiave viene usata per metodi che si applicano ad una classe e non ad un singolo oggetto (ovvero il metodo non lavora con le proprietà degli oggetti, ma solo con i propri parametri).
Un metodo non può essere definito statico se lavora con proprietà non statiche della classe.
I metodi statici non vengono trattati con il polimorfismo.
I metodi statici non si applicano ad oggetti.
es. Math.max() //*NomeClasse.nomemetodo()*

Confronto tra oggetti

Gli operatori == e != verificano se 2 oggetti puntano alla stessa locazione di memoria.
il metodo equals() verifica se due stringhe siano uguali.

Incapsulazione
Il compito dell'incapsulazione è nascondere le parti interne di un oggetto che servono solo per realizzarne le funzionalità, permettendo l'accesso all'oggetto e al suo uso solo attraverso strumenti definiti dal programmatore.
Le 4 p della protezione sono:

- **public**: metodi e campi pubblici sono utilizzabili o richiamabili anche all'esterno del package o della classe
- **protected**: metodi e campi protetti possono essere utilizzati da tutte le classi del package (dalle sottoclassi anche fuori dal package)
- **package**: metodi e campi sono accessibili alle classi nello stesso package (metodo di default)
- **private**: metodi e campi privati possono essere utilizzati o rinominati solo all'interno della stessa classe.

Una proprietà privata esiste, ma non può essere vista; è possibile manipolarla con i metodi setProprietà(tipo valore); getProprietà().
In generale si dichiarano i campi (proprietà) privati e si manipolano tramite metodi di accesso (pubblici).

Gli scopi dell'incapsulazione sono:

1. **protezione dati**: il metodo setter controlla che il valore assegnato sia corretto (una variabile condivisa tra più thread può essere modificata solo da metodi sincronizzati pubblici)
2. **information hiding**: certe proprietà non possono essere modificate liberamente ma solo in casi specifici (si definisce il metodo getter); certi metodi vanno richiamati con parametri fissi (metodi ricorsivi)
3. **astrazione**: nel momento in cui vengono definite le intestazioni dei metodi pubblici per la manipolazione di un oggetto, ne posso modificare l'implementazione senza che questo influenzi il resto del programma (*mostra gli attributi essenziali e nasconde le informazioni irrilevanti*)
4. **confinamento errori**: L'assenza di effetti collaterali (come modifiche permanenti all'ambiente esterno al metodo) diminuisce la possibilità di errori

this
La parola chiave this rappresenta un parametro implicito per i metodi di istanza (i metodi non statici); this permette di riferirsi all'oggetto corrente (cioè all'oggetto a cui il metodo è applicato o all'oggetto che un metodo costruttore restituirà).

Passaggio dei parametri
I parametri formali sono quelli usati per definire un metodo, mentre i parametri attuali sono quelli che si indicano quando il metodo viene richiamato.

In Java i parametri sono passati per valore. Quando un metodo viene richiamato:
1) i parametri attuali vengono valutati
2) i valori ottenuti vengono assegnati al corrispondente parametro formale

Esiste un modello di oggetti per riferimento in cui, mentre i puntatori agli oggetti sono passati per valore, gli oggetti puntati (ovvero l'area di memoria) sono passati per riferimento.
Un metodo può:
1) modificare le proprietà dell'oggetto che gli viene passato
2) modificare le proprietà dell'oggetto a cui viene applicato (utilizzando implicitamente o esplicitamente per il parametro this)

Overloading
E' possibile, all'interno della stessa classe, creare metodi con lo stesso nome, ma con differenti intestazioni e definizioni.
Il numero e il tipo degli argomenti permettono di distinguere i metodi.

Classi e metodi astratti
Una classe astratta ha lo scopo di definire caratteristiche comuni alle sottoclassi; non è possibile istanziare oggetti di una classe astratta. I metodi astratti hanno intestazione, ma non implementazione (vengono implementati nelle sottoclassi); possono solo essere definiti in classi astratte.
Un metodo astratto soddisfa la pretesa del compilatore che il metodo richiamato sia definito in una classe.

4 COMPLESSITA'

La complessità computazionale valuta le risorse necessarie (tempo di calcolo e memoria) per l'esecuzione di un dato algoritmo e studia le risorse minime necessarie per la risoluzione di un problema.
Il costo di un algoritmo viene valutato in funzione della dimensione del problema. (valore proporzionale all'occupazione in memoria dei dati del problema)
ES

1. per un problema sugli interi il numero delle cifre dell'intero (per un problema sugli interi il valore n dell'intero)
   n cifre ↔ b ^ n
   v ↔ numero cifre = log b v
2. per un problema sugli array la lunghezza dell'array
3. per un problema sulle liste la lunghezza della lista
4. per un problema sugli alberi il numero di elementi dell'albero

Esistono problemi:

1. facili, di cui si conoscono algoritmi di risoluzione efficienti
2. difficili, di cui gli unici algoritmi noti non sono efficienti (oppure quelli per cui è dimostrato che algoritmi efficienti non esistono)

Una misurazione dettagliata può essere difficilmente applicabile; per questo si introducono delle approssimazioni che danno una limitazione dall'alto oppure dal basso di una funzione.
In particolare si ricorre alla notazione O(f(n)):
$g(n) \in O(f(n))$ se $\exists$ (n0, c), $c \geq 0$, $n0 \geq 0$ tali che $\forall$ $n > n0$, $g(n) \leq c * f(n)$
Si dice quindi che un algoritmo opera in tempo O(f(n)) se termina in un tempo proporzionale a f(n) dato un input di dimensione n.

In questo ambito tornano dunque utili altre due misure, complementari della notazione O grande:

- $g(n)=\Omega(f(n))$ se $\exists$ (n0,c) tali che $\forall n>n0 g(n)\geq cf(n)$, per $c>0$, $n0\geq 0$. Cioè g(n) cresce non più lentamente di f(n); questa notazione è utile per valutare il caso ottimo di un algoritmo: se un algoritmo è $\Omega(f(n))$ ("Omega di f(n)") significa che nel caso migliore richiede f(n) passi per essere risolto.
- $g(n)=\Theta(f(n))$ se $g(n)\in O(f(n))$ e $g(n)\in\Omega(f(n))$, cioè g(n) cresce altrettanto rapidamente di f(n). Se un algoritmo è $\Theta(f(n))$ ("Theta di f(n)"), non ci sono variazioni significative di prestazioni tra il caso migliore e il caso peggiore.

Le prestazioni di un algoritmo possono dipendere, oltre che dalla dimensione del problema, anche da come si presentano i dati in input; si considerano:

1. Il caso migliore è il caso in cui i dati sono i migliori dati possibili per l'algoritmo, cioè quelli che richiedono meno elaborazioni per essere trattati.
2. Il caso peggiore invece prevede i dati che richiedono il massimo numero di passi per l'algoritmo.
3. Il caso medio è il caso più utile da analizzare perché fornisce un reale indicatore della complessità dell'algoritmo ma tendenzialmente è anche quello più complesso dato che spesso è difficile determinare quali sono i dati medi.

Costo computazionale
Il costo computazionale di un programma è un costo definito in termini di risorse di calcolo.
Le risorse di calcolo fondamentali sono due:

1. quantità di tempo necessario alla computazione (tempo)
   Quante chiamate di metodo in contemporanea ho al massimo?
2. quantità di memoria utilizzata (spazio)
   Quanta memoria serve in una singola chiamata?

Il costo dipende dal tipo di applicazione:

1. **iterazione**: Tempo -> 1 chiamata di metodo//costante
   numero di operazioni: quanti cicli faccio?
   quante operazioni per ogni ciclo? -> numero costante
   Spazio -> Memoria costante (parametri e variabili locali)
2. **Ricorsione**: Tempo -> quante chiamate ricorsive ***<u>in tutto</u>*** faccio?
   numero di operazioni: quante operazioni per ogni chiamata? -> costante
   Spazio -> quante chiamate ricorsive ***<u>in contemporanea</u>*** faccio al massimo?
   Memoria costante (parametri e variabili locali)

<u>5 RICORSIONE</u>
La ricorsione è una tecnica per risolvere problemi complessi riducendoli a problemi dello stesso tipo, ma più semplici (ovvero, su dati di dimensione inferiore).
Per risolvere un problema ricorsivamente occorre individuare una dimensione del problema tale che:

1. per il valore più basso, la soluzione possa essere espressa direttamente (senza usare la ricorsione)
2. sia possibile risolvere il problema per la dimensione generica supponendo di saper risolvere il problema per dimensioni inferiori
3. un metodo ricorsivo contenga un parametro che identifica la dimensione del problema su cui il metodo deve lavorare.

La dimensione è un valore proporzionale in memoria dei dati del problema.

Come si scrive un metodo ricorsivo?

1. Un parametro esprime la dimensione del problema e la posizione
2. un if per controllare se ci si trova nel caso terminale, ovvero se il parametro ha il valore più basso possibile (in questo caso si esprime direttamente il risultato):
3. la chiamata ricorsiva permette di risolvere lo stesso problema, usando lo stesso modulo, ma su un insieme di dati più piccolo.
4. Sapendo quanto vale il risultato sul sottoproblema (su dati più piccoli), si calcola quanto vale il risultato del problema originale.

La ricorsione è molto comoda in situazioni in cui per risolvere il problema è necessario fare più chiamate ricorsive (ad esempio 2) (quicksort, torri di Hanoi, algoritmi sugli alberi binari).
Per calcolare F (n):

- se n e' un caso base, riporta la soluzione per il caso n
- altrimenti: risolvi i problemi più semplici F (n1),F (n2), ...F (nk) (*chiamate ricorsive*)
- combina le soluzioni ottenute e riporta combina(F (n1),F (n2), ...F (nk))

Un processo ricorsivo termina se le chiamate ricorsive si avvicinano ai casi di base: dopo un numero finito di chiamate ricorsive si arriva a casi base.
Ma come varia il tempo di esecuzione al variare della dimensione dei dati?

Dipende dalla complessità dell'algoritmo:

1. *Complessità logaritmica: (ricerca di un elemento in vettore ordinato)*
   dimensione problema n → Tn = log n
   dimensione problema c * n → Tc*n = log (c*n) = log c + log n = Tn + log c
   un problema c volte più grande consuma un tempo di poco più grande.
2. *Complessità lineare: (somma degli elementi di un vettore, fattoriale)*
   dimensione problema n → Tn = n
   dimensione problema c * n → Tcn = c*n = c * Tn
   un problema c volte più grande consuma un tempo c volte più grande
3. *Complessità polinomiale: (moltiplicazione di due matrici)*
   dimensione problema n → Tn = n^k
   dimensione problema c * n → Tcn = (c*n)^k = c^k * n^k = c^k * Tn
   un problema c volte più grande consuma un tempo ck volte più grande
4. *Complessità esponenziale: (Torri di Hanoi)*
   problema di dimensione n → Tn = 2n
   problema di dimensione n+1 → Tn+1 = 2n+1 = 2 *2n = 2* Tn
   un problema di una unità più grande consuma un tempo doppio

Esempio di fattoriale ricorsivo di coda
Aggiungiamo un parametro che rappresenta il risultato delle chiamate ricorsive precedenti; nel caso terminale restituiamo il valore di questo parametro.
Questo parametro viene inizializzato nella prima chiamata.

```
private static long factRC(int n, long ris){
        if (n == 0) return ris;
        else return fact (n-1, ris*n);
}
public static long factRC(int n){ return factRC(n,1); }
```

Numeri di Fibonacci

```
static long fiboR (int n) {
        if (n<=2) return 1;
        else return fiboR(n-1) + fiboR(n-2);
}
```

La complessità spaziale della funzione è O(n): ci sono al massimo n chiamate attive contemporaneamente.
La complessità temporale della funzione è O(2^n). Infatti la funzione fibon(n) richiamerà al suo interno fibon(n-1) e fibon(n-2); fibon(n-1) richiamerà al suo interno fibon(n-2) e fibon(n-3). Entrambe le funzioni fibon(n-2) richiameranno al loro interno fibon(n-3) e fibon(n-4) e così via.

Elevamento a potenza

```
public static double potenzaR2(double b, long e){
	if (e==0) return 1;
	else if (e % 2==0) return potenzaR2(b*b, e/2);
	else return b* potenzaR2(b*b, e/2);
}
```

Numero di chiamate ricorsive pari al numero di volte che è possibile dividere l'esponente per 2, per ogni chiamata un numero costante di operazioni -> complessità temporale Teta(log(esponente))
Massimo numero di chiamate ricorsive attive contemporaneamente pari al log2(esponente) -> complessità spaziale Teta(log(esponente))

Ricerca binaria
Si tratta di ricercare un elemento all'interno di un vettore ordinato.
Sfruttiamo l'ordinamento e l'accesso casuale (il costo dell'accesso ad un elemento non dipende dalla posizione).
Supponiamo di aver definito la classe Vettore che contiene al suo interno la proprietà dati (un array di interi). Se l'array dati è ordinato la ricerca di un elemento può essere fatta utilizzando l'algoritmo di ricerca binaria.
Consideriamo tutto il vettore: prendiamo l'elemento di mezzo. Se questo è l'elemento che cerchiamo la ricerca è finita.
Se l'elemento che cerchiamo è più grande dell'elemento centrale del vettore proseguiamo la ricerca nella metà superiore, altrimenti proseguiamo nella metà inferiore. Ci fermiamo quando troviamo l'elemento cercato o quando la porzione di vettore da esaminare diventa vuota: in questo caso vuol dire che l'elemento cercato non è presente nel vettore.

```
public class Vettore {
	int [] dati;
	public int ricercaBinRic (int x){ return ricercaBinRic(dati, x,0,dati.length-1);}
	//parametri attuali
	private static int ricercaBinRic (int [] dati, int x, int inizio, int fine){ //parametri formali
		if (inizio > fine) return -1;
		else {
			int centro = (inizio+fine)/2; // inizio + (fine-inizio)/2
			int c = dati[centro];
			if (c==x) return centro;
			else if (x < c ) return ricercaBinRic(dati, x,inizio,centro-1);
			else return ricercaBinRic(dati, x,centro+1,fine);
		}
	}
}
```

Quante chiamate faccio al massimo in contemporanea? log2 (N)
Quanta memoria occupo per ogni chiamata? 3 par. Int + 2 var loc int + 1 oggetto vettore (cioè un riferimento (cioè un indirizzo di memoria) esprimibile con 4 o 8 byte) < C
Quante chiamate faccio in totale? (nel caso peggiore) log (N) chiamate per ogni chiamata faccio un numero costante di operazioni -> **complessità** Teta(log N)

Versione iterativa ricerca binaria

```
public static int ricercaBinIter (int dati[], int x){
	int inizio=0, fine= dati.length-1;
	while (inizio <= fine) {
		int centro = (inizio+fine)/2; //inizio+ (fine-inizio)/2
		int c = dati[centro];
		if (c==x) return centro;
		else if (x < c ) fine=centro-1;
		else inizio=centro+1;
	}
	return -1;
}
```

Costo: memoria: costante operazioni: Teta(log N)

Algoritmo quicksort

Possiamo realizzare un algoritmo per l'ordinamento di un vettore.
Dimensione del problema: numero di elementi vettore da ordinare
Se il vettore contiene un solo elemento (oppure è vuoto) allora il vettore è già ordinato altrimenti:

1. scegliamo un elemento del vettore (per esempio il primo); sia esso X; permutiamo gli elementi del vettore in modo da portare nella parte iniziale del vettore tutti gli elementi più piccoli di X e nella parte finale del vettore tutti gli elementi più grandi di X;
2. ordiniamo la parte iniziale del vettore (che ora contiene gli elementi più piccoli di X)
3. ordiniamo la parte finale del vettore (che ora contiene gli elementi più grandi di X)

Il punto 2. ed il punto 3. sono lo stesso problema dell'ordinamento, ma applicati ad un vettore più piccolo: sono risolvibili tramite una chiamata ricorsiva.

1. Metodo partition

Dato un vettore (di interi tanto per fissare le idee) di cui si considera la parte che va dalla posizione inizio alla posizione fine, fissa un elemento detto pivot (perno) permuta gli elementi in modo da portare tutti gli elementi più piccoli del perno alla sua sinistra e gli elementi più grandi del perno alla sua destra. Restituisce la posizione finale del perno.

```
static int partition(int[] array, int inizio, int fine) {
      int pivot = array[fine];
      int i = inizio - 1;

      // Traverse through all elements
      // If element is smaller than or equal to pivot, swap it
      for (int j = inizio; j < fine; j++) {
         if (array[j] <= pivot) {
            i++;
            // Swap elements at i and j
            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
         }
      }
      // Swap the pivot element with the element at i+1
      int temp = array[i + 1];
      array[i + 1] = array[fine];
      array[fine] = temp;

      // Return the partition index
      return i + 1;
   }
```

2. e 3.

L'algoritmo di ordinamento è dunque:

```
private static void quicksort (int v[], int inizio, int fine) {
          if ( inizio < fine) {
                    int mid = partition(v, inizio, fine);
                    quicksort(v,inizio,mid-1);
                    quicksort(v,mid+1,fine);
          }
}
public void quicksort () { quicksort (v, 0, v.length-1); }
```

Per valutare la complessità di quicksort considereremo il numero di confronti effettuati.
All'interno della procedura partition ogni elemento viene confrontato con l'elemento X che fa da perno: vengono quindi effettuati fine-inizio+1 confronti.
Per quanto riguarda il numero di chiamate ricorsive vediamo che il caso migliore è quello in cui ad ogni passo l'insieme viene diviso in due sottoinsiemi di dimensioni uguali. In questo caso sono sufficienti log2 N livelli di ricorsione.
Il caso più sfavorevole è quando X è sempre l'elemento più grande o quello più piccolo dell'insieme: in questo caso l'insieme viene diviso in un insieme di un elemento ed in un insieme di n-m elementi. In questo caso saranno necessari N livelli di ricorsione.

Nel caso più favorevole la **complessità temporale** risulta dunque di ordine Teta(Nlog2 N).
Nel caso più sfavorevole la **complessità** è di ordine O(N^2): se scegliamo come pivot il primo elemento della parte del vettore da ordinare, il caso peggiore risulta essere quando l'insieme è già ordinato, non importa se in senso crescente o decrescente.
Per quanto riguarda la **complessità spaziale**, il numero di chiamate attive contemporaneamente è di ordine O(N) nel caso peggiore (che, per come abbiamo scritto la partition, si presenta ad esempio se l'insieme è già ordinato), mentre è O(log N) nel caso migliore.
In situazioni pratiche, per evitare di ricadere nel caso peggiore, si può scegliere la posizione del pivot a caso ad ogni passo. In questo modo è praticamente impossibile cadere nel caso peggiore.

Implementazione iterativa quicksort
Definiamo una struttura a pila in cui immagazzinare le parti del vettore ancora da ordinare: in questo modo simuliamo il meccanismo di attivazione e disattivazione degli ambienti effettuato automaticamente dalla ricorsione.
La pila è una struttura dati gestita con politica LIFO (Last In First Out): l'elemento estratto è l'ultimo che è stato inserito. La pila la realizzeremo concretamente con una lista in cui faremo inserimenti ed estrazioni in testa. La proprietà "*lunghezzamassima*" misura il massimo numero di elementi che la pila ha contenuto durante l'elaborazione.

Il metodo partition è esattamente lo stesso visto prima.
L'implementazione non ricorsiva di quicksort diventa:

1) inserisco nella pila l'intervallo [0, lunghezza vettore-1]
2) while (l'elemento e che estraggo dalla pila non è nullo){
   eseguo la partition sull'intervallo descritto da *e*
   inserisco nella pila i 2 sottointervalli prodotti dalla partition (non inserisco l'elemento se indica un vettore vuoto o un vettore con un solo elemento) (per ottimizzare la dimensione della pila inseriamo per ultimo l'intervallo più piccolo, così che sia elaborato per primo) }

Altro sul quicksort
L'algoritmo Quicksort è un esempio particolare di un modo generale di impostare un algoritmo, detto *divide et impera*. Esso consiste nel dividere i dati, risolvere ricorsivamente il problema sui sottoinsiemi ed ottenere la soluzione globale come combinazione delle soluzioni parziali. Tale tecnica può portare vantaggi in termini di complessità, purché le dimensioni dei sottoinsiemi dei dati non siano troppo differenti fra loro (e la loro somma non supera la dimensione iniziale).

Torri di Hanoi

Si consideri la seguente situazione: siano dati tre pioli sul primo dei quali, detto sorgente, siano infilati N dischi di diametro differente, ordinati dal più grande (che si trova in basso), al più piccolo. Gli altri due pioli non hanno nessun disco infilato e si dicono intermedio e destinazione. Il problema delle torri di Hanoi può così enunciato: trasferire gli N dischi dalla posizione sorgente alla posizione destinazione, utilizzando la posizione intermedia, muovendo un solo disco alla volta e non sovrapponendo mai un disco di dimensione maggiore ad un disco di dimensione minore.

Se N=1 basta collocare il disco nel piolo destinazione.

Se invece N > 1 basta collocare i primi N-1 dischi nel piolo intermedio, utilizzando il piolo destinazione. A questo punto si può portare il disco N nel piolo destinazione. Infine si portano gli N-1 dischi dal piolo intermedio nel piolo destinazione, utilizzando il piolo sorgente. L'algoritmo risulta dunque essere:

```
if (N==1)
        1. trasferisci il disco N da S in D;
else {
        2. trasferisci uno alla volta N-1 dischi da S in I utilizzando D e mantenendo
        l'ordinamento;
        3. trasferisci il disco N da S in D;
        4. trasferisci uno alla volta N-1 dischi da I in D utilizzando S e mantenendo
        l'ordinamento;
}
```

1. e 3. possono essere eseguiti direttamente.

2. e 4. sono equivalenti al problema iniziale avendone però diminuito la dimensione (cioè il numero di dischi), e la funzione dei tre pioli S, I e D.

```
public void hanoi (int N) { hanoi (N, "sorgente", "intermedio", "destinazione"); }
private void hanoi (int N, String S, String I, String D) {
        if (N == 1) trasferisci (1,S,D);
        else {
                hanoi (N-1, S,D,I);
                trasferisci (N,S,D);
                hanoi (N-1, I,S,D);
        }
}
private void trasferisci (int N, String S, String D) {
        System.out.println("Sposta il disco "+N + " da "+S+" a "+D);
}
```

Per valutare l'ordine di **complessità** basta contare il numero di chiamate ricorsive, osservando che per ogni chiamata viene effettuato un solo trasferimento. Ogni procedura richiama due procedure sullo stesso numero di nodi meno uno. La procedura hanoi(N) richiamerà ricorsivamente due volte hanoi(N-1). Ciascuna delle hanoi(N-1) richiamerà due volte hanoi(N-2) e così via.

Al massimo ci sono N chiamate ricorsive attive contemporaneamente. -> **complessità spaziale** O(N)

In tutto ci saranno quindi state 2^0+2^1+2^2+ ...+2^N-1 = 2^N -1 chiamate ricorsive. La **complessità temporale** risulta quindi O(2^N).

Bubblesort
Il bubblesort è un altro algoritmo per l'ordinamento di un vettore.
Facciamo n passate: ad ogni passata incremento la parte ordinata del vettore.
N-1 passi: ad ogni passo porto l'elemento più piccolo della parte non ordinata all'inizio della parte non ordinata. Come? Partendo da fondo considero gli elementi a coppie: se sono nell'ordine sbagliato li scambio.

```
static public void ordina(int vettore[]) { //N elementi
	int temp;
	for (int i = 0; i < vettore.length - 1; i++) { // N-1 passi
		for (int j= vettore.length-1 ; j > i; j--) { // N-(i+1) passi
			if (vettore[j] < vettore[j-1]) { // 1 confronto
				temp = vettore[j];
				vettore[j] = vettore[j - 1]; //0 o 3 assegnamenti
				vettore[j - 1] = temp;
			}
		}
	}
}
```

Complessità spaziale: costante (vettore è un riferimento!!)

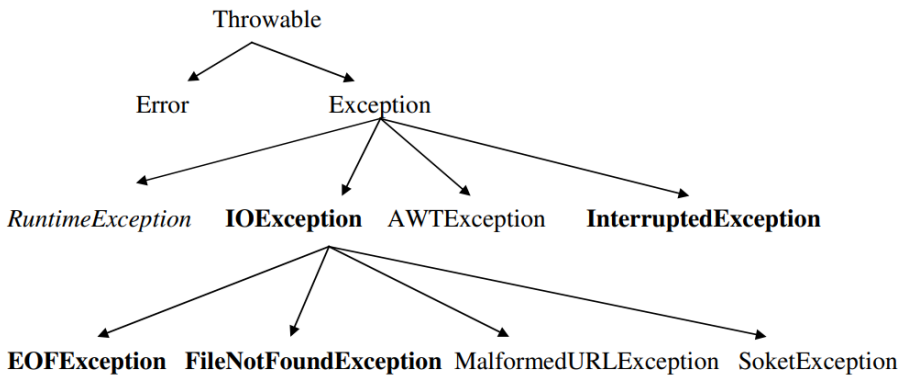
Ci sono due cicli for l'uno dentro l'altro. Sia N la lunghezza del vettore.
Il ciclo for più esterno viene eseguito N-1 volte, quello più interno N-1-I volte.
Contiamo ora il numero di operazioni. Per ogni iterazione del ciclo for più esterno vengono eseguiti N-I-1 confronti quindi in totale ne vengono eseguiti: (N-1)+(N-2)+...+1 = N(N-1)/2.

Per quanto riguarda gli assegnamenti, nel caso migliore gli assegnamenti che sono situati nel ciclo for più interno, non vengono mai eseguiti (perché A[J]<A[J-1] è sempre falso).
Nel caso peggiore invece, per ogni iterazione del ciclo for interno vengono eseguiti 3 assegnamenti, quindi in tutto ne vengono eseguiti 3(N-1)+3(N-2)+...+3 = 3N(N-1)/2.

caso migliore (insieme già ordinato) N(N-1)/2 confronti
caso peggiore (insieme inversamente ordinato) N(N-1)/2 confronti + 3N(N-1)/2 assegnamenti.
Complessità temporale O(N^2)
Mediamente si dimostra che l'algoritmo si comporta come nel caso peggiore.

6 ECCEZIONI

L'eccezione è un accadimento anomalo (o inaspettato) che avviene durante l'esecuzione del programma. Le eccezioni vengono sollevate (lanciate), in risposta ad un evento inusuale; è possibile sollevare esplicitamente una eccezione.
Un'eccezione è un oggetto di una classe che discende dalla classe Throwable.



RuntimeException (e sue sottoclassi): non controllate.
Alcune sottoclassi di *RuntimeException*: NullPointerException, ClassCastException, IndexOutOfBoundsException, **NumberFormatException** (deve essere controllata).
Exception (e altre sue sottoclassi): controllate (cioè è obbligatorio richiamare i metodi che possono lanciare questo tipo di eccezioni all'interno di un blocco try catch).

Protezione dalle eccezioni

```
try { //codice da proteggere
        i1; //apro un file
        i2; // lo leggo
        i3; //elaboro i dati del file
        i4;
}
catch (EccezioneTipo1 e1) { i5;...} //FileNotFoundException
catch (EccezioneTipo2 e2) {i6;... } //IOException
catch (EccezioneTipo3 e3) {i7; ...} //Exception
finally {i8; . . . } // chiusura del file
i9;
```

Se un metodo solleva una eccezione l'esecuzione del blocco try viene interrotta e si passa ad eseguire il primo catch il cui parametro è compatibile con l'eccezione sollevata.
Poi si esegue il blocco finally, se è presente.
L'esecuzione prosegue con la prima istruzione che segue il try-catch-finally.
Non è obbligatorio gestire con un try-catch i metodi che possono sollevare eccezioni che discendono da Error o da RuntimeException.
I blocchi catch vanno messi in ordine, da quello che riguarda l'eccezione più specifica a quello che riguarda l'eccezione più generica; i catch vengono esaminati in ordine: il primo catch il cui parametro è compatibile con l'eccezione che è stata generata cattura l'eccezione; se sbaglio l'ordine il compilatore se ne accorge.
Un blocco try catch si può trovare anche all'interno di un blocco finally.

Dichiarazione di metodi che possono sollevare eccezioni

```
public boolean metodo2 (int x) throws Eccezione1, Eccezione2, Eccezione3 { . . .}
```

in questo caso non è necessario inserire l'istruzione read in un blocco try catch: le eccezioni vengono passate al metodo chiamante, che le deve gestire. Quando un metodo viene ridefinito è possibile sollevare meno eccezioni del metodo originale, ma non di più.

```
public void leggiFile (String nomefile) throws IOException { }
public static void main (String a[]){
        try { leggiFile ("pippo.txt");}
        catch ( IOException e ) {System.out.println(e.getMessage());}
}

public static void main (String a[]) throws IOException {
        //l'eccezione viene passata all'ambiente di esecuzione che la gestisce con la propria
        azione di default
        leggiFile ("pippo.txt");
]
```

Creazione e lancio di eccezioni

Le eccezioni sono classi, quindi è possibile derivare nuove eccezioni.

Creazione:

```
public class SunSpotException extends Exception{
        public SunSpotException(){ }
        public SunSpotException(String msg){super(msg); }
}
```

Lancio di una eccezione all'interno di un metodo:

```
public String leggiMessaggio(Socket s) throws SunSpotException{ …
        if (….) throw new SunSpoException("Errore: comunicazione disturbata");
}
```

Quando un'eccezione viene lanciata il metodo termina immediatamente (ma se c'è una clausola finally il suo codice viene eseguito) e nessun valore viene restituito.
Se il metodo chiamante non ha il blocco try-catch a proteggere la chiamata, il programma potrebbe terminare.

7 FLUSSI DI INPUT E OUTPUT

I flussi di I/O in Java sono canali attraverso i quali i dati possono essere trasferiti da una fonte (input) a una destinazione (output). Questi flussi possono essere utilizzati per leggere dati da un file, scrivere dati su un file, o persino comunicare con altri programmi o dispositivi di rete. Le classi messe a disposizione da java sono incluse nel pacchetto java.io.
Java offre due tipi principali di flussi:

- **Flussi di Byte**: utilizzati per gestire dati binari, come immagini, audio, video, o qualsiasi altro tipo di dato non testuale.
- **Flussi di Caratteri**: utilizzati per gestire dati testuali, come file di configurazione, log, o documenti.

Flussi di byte

I flussi di byte sono progettati per gestire dati binari. Le classi principali per i flussi di byte sono *InputStream* e *OutputStream*, che sono classi astratte e rappresentano rispettivamente un flusso di input e output di byte. Esempi di classi concrete:

- **FileInputStream** e **FileOutputStream**: utilizzate per leggere e scrivere file binari.
  FileOutputStream s = new FileOutputStream(nomefile);
- **BufferedInputStream** e **BufferedOutputStream**: migliorano le prestazioni delle operazioni di I/O utilizzando un buffer interno.
  BufferedInputStream s=new BufferedInputStream (new FileInputStream(nomefile));
- **DataInputStream** e **DataOutputStream**: permettono di leggere e scrivere tipi di dati primitivi (come int, double, boolean, ecc.) in modo diretto.
  Metodi di DataInputStream: readBoolean(), readByte(), readInt(), …, readLine()
  Metodi di DataOutputStream: writeBoolean(),..; writeBytes(String), writeChars(String)

infile = new DataInputStream(new BufferedInputStream(new FileInputStream(nome)))

- **ObjectInputStream** e **ObjectOutputStream**: consentono l'input ed output di oggetti
  Metodi di ObjectInputStream: quelli di DataInputStream più readObject ()

Flussi di caratteri

I flussi di caratteri sono progettati per gestire dati testuali. Le classi astratte principali per i flussi di caratteri sono *Reader* e *Writer*, che rappresentano rispettivamente un flusso di input e output di caratteri. Esempi di classi concrete:

- **FileReader** e **FileWriter**: utilizzate per leggere e scrivere file di testo.
  FileWriter r = new FileWriter(new File(nomefile));
- **PrintWriter**: Metodi print(), println()
- **BufferedReader** e **BufferedWriter**: migliorano le prestazioni delle operazioni di I/O utilizzando un buffer interno.
  BufferedReader implementa il metodo readLine()
  BufferedWriter implementa il metodo NewLine(line)
  BufferedReader r = new BufferedReader(new FileReader(nomefile));
- **InputStreamReader** e **OutputStreamWriter**: fungono da ponte tra i flussi di byte e i flussi di caratteri.
- **Scanner**: Permette l'input formattato di flussi di caratteri
  Metodi: nextBoolean(), hasNextBoolean(), …, nextLine(), hasNextLine()

Chiusura dei flussi
Le risorse di I/O, come i flussi di file, devono essere chiuse correttamente dopo il loro utilizzo per evitare perdite di risorse e potenziali problemi di sistema. A partire da Java 7, è stato introdotto il costrutto try-with-resources, che garantisce la chiusura automatica delle risorse, anche in caso di eccezioni.

Schema
//Uso Scanner -> non metto try catch nel main (nextLine, hasNextLine)
//tranne se non restituisco niente nel caso di errore, in quel caso mi comporto come BufferedReader

- FileReader (nomefile) -> Scanner (FileReader)
- String nome = in.nextLine(); o int n = in.nextInt();
- while (in.hasNextLine)
  if (in.hasNextInt)

//Uso BufferedReader -> metto try-catch nel main e throw nel metodo (readLine, Integer.parseInt)

- FileReader (nomefile) -> BufferedReader (FileReader)
- while ((String riga = in.readLine()) != null)
  int n = Integer.parseInt(riga); //la stringa diventa numero
  metto NumberFormatException

<u>8 MULTITHREADING</u>
Processo: esecuzione di un programma all'interno del suo proprio spazio di indirizzi.
Thread: singolo flusso di esecuzione all'interno di un processo.
Più Thread sono usati in particolare per effettuare più attività contemporaneamente.

Creazione e uso di thread
Due modi per la creazione:

1. si deriva la classe dalla classe Thread
```
public class ServerChatThread extends Thread{
        public void run(){. . . }
} public static void main (String args []){
        Oggetto o = new Oggetto();
        ServerChatThread t1 = new ServerChatThread(o);
        o.metodo();
        istruzione0;
        t1.start(); istruzione1; istruzione2; …; t1.suspend(); t1.resume(); t1.interrupt();
}
```
2. si implementa l'interfaccia Runnable
```
public class Animazione extends JFrame implements Runnable{
        public void run(){. . . }
} public static void main (String args []){
        Animazione aut = new Animazione();
        Thread t1 = new Thread(aut); t1.start(); … }
```

yield(): cede il passo ad un altro thread
t.join(): si aspetta che il thread t abbia terminato
Thread.sleep(...): sospende momentaneamente le operazioni in corso

Sincronizzazione
Quando più thread accedono allo stesso dato di cui almeno uno in modifica è necessario usare la sincronizzazione per evitare il rischio di modifiche inconsistenti.
La sincronizzazione ha lo scopo di assicurare che un solo thread alla volta acceda al dato.

Non si può essere sicuri di come le micro operazioni svolte dai thread siano svolte. Questo può provocare delle modifiche inconsistenti ai dati nel caso in cui più thread accedono in modifica allo stesso dato.
Un ambiente thread-safe consente ai thread di coesistere senza conflitti.
L'uso della sincronizzazione tramite la parola chiave ***synchronized*** permette di gestire situazioni in cui più thread accedono in modifica allo stesso dato.
Quando un thread esegue l'istruzione: *oggetto.metodo()*; e *metodo* è stato dichiarato ***synchronized*** avviene la seguente cosa:
1) il thread verifica se oggetto è libero; se sì lo blocca (*lock*) ed esegue il metodo; altrimenti viene posto in attesa;
2) quando il thread ha concluso l'esecuzione di metodo(), l'oggetto viene sbloccato (*unlock*). Se ci sono thread in attesa di eseguire oggetto.metodo() uno di loro viene riattivato.
L'uso della sincronizzazione è necessario per evitare modifiche inconsistenti dei dati, ma può provocare altri problemi: colli di bottiglia e deadlock.

Un thread sincronizzato può eseguire la chiamata del metodo *wait()* su un oggetto: in questo modo la sua esecuzione viene sospesa;
quando un altro thread richiama il metodo *notifyAll()* su un oggetto monitor risveglia tutti i thread che erano stati sospesi su quel monitor;
quando un altro thread richiama il metodo *notify()* su un oggetto monitor risveglia uno a caso dei thread che erano stati sospesi su quel monitor.

9 PROGRAMMAZIONE DI RETE

Connessioni tramite socket

Il socket è un elemento astratto del software di rete che consente al programma di ricevere e trasmettere dati.
Un socket è concettualmente una porta che collega un certo processo al mondo esterno.
E' identificata da un numero (port number) unico su una data macchina (nodo o host).
Ogni nodo e' identificato dal suo indirizzo IP.
Permette a due processi, residenti sulla stessa macchina o anche molto distanti, di comunicare fra loro nello stesso modo.
Modello client / server:

1. il server deve stare in attesa di possibili comunicazioni in arrivo
2. i client (anche più di uno) parlano con il server (enti attivi)...

SocketStream

Le socket stream sono affidabili, perché stabiliscono una connessione stabile e bidirezionale con l'altra parte, che dura finché non si decide di chiuderla.
Si usa quando l'ordine dei messaggi è importante e l'affidabilità è cruciale.
Spesso c'è un limite massimo alle connessioni che si possono aprire.

Il server crea un ServerSocket con un numero e si mette in attesa. Un client, quando vuole comunicare con il server, crea la sua Socket specificando con chi vuole parlare (nome dell'host, numero di porta). Il server accetta la richiesta del client: con ciò si crea una Socket già collegata al client, tramite cui i due comunicano. Al Socket sono associati due stream, uno dal client verso il server ed uno dal server verso il client.
La comunicazione client/server è bidirezionale: i ruoli "client" e "server" sono tali solo nella fase iniziale, quando si instaura la connessione; una volta connessi, i due processi si parlano alla pari Il ServerSocket serve per stare in attesa di richieste dai client: quando ne arriva una, l'effettiva comunicazione avviene tramite un nuovo Socket appositamente creato, quindi il ServerSocket si libera. E' però necessario manualmente rimetterlo in attesa di altri client.

Costruire una Socket significa aprire la comunicazione verso l'altra parte.
public Socket(InetAddress remoteAddr, int remotePort) crea una socket stream e la collega alla porta specificata della macchina remota corrispondente all'indirizzo IP dato:
public Socket(String remoteHost, int remotePort) crea una socket stream e la collega alla porta specificata della macchina remota corrispondente al nome dato.

Per comunicare, si recuperano dalla socket i due stream (di ingresso e di uscita) a essa associati, tramite i metodi:
public InputStream getInputStream() restituisce lo stream di input da cui leggere i dati (byte) che giungono dall'altra parte
public OutputStream getOutputStream() restituisce lo stream di output su cui scrivere i dati (byte) da inviare all'altra parte Poi, su questi stream si legge / scrive come su qualunque altro stream di byte.
Al termine, per chiudere la comunicazione si chiude il Socket: *public synchronized void close()* chiude la connessione e libera la risorsa.

Costruire una ServerSocket significa predisporsi a ricevere richieste di connessione da clienti remoti.
public ServerSocket(int localPort) crea una ServerSocket e la collega alla porta locale specificata
public ServerSocket(int localPort, int count) crea una ServerSocket che accetta al massimo count richieste pendenti, e la collega alla porta locale specificata
public Socket accept() mette il server in attesa di nuove richieste di connessione: se non ce ne sono, il server si blocca in attesa, altrimenti restituisce un oggetto Socket, tramite cui avviene l'effettiva comunicazione tra client e server. Da quel momento in poi, anche il server comunica col client tramite una normale Socket.

SERVER

1. Creo ServerSocket ss = new ServerSocket (indirizzo)
2. Ciclo true in un try
    a. Server s = ss.accept();
    b. ThreadServer ts = new ThreadServer (s);
3. ts.start();

CLIENT

1. String server;
2. if(args.length == 1){server = args[0];}
3. else{server = "localhost"; }

CLIENT MANDA COSE AL SERVER (dentro try)

1. CLIENT (SCRITTURA NUMERI) Socket = new socket (server, indirizzo) -> OutputStream = Socket.getOutputStream; -> DataOutputStream (writeInt, flush).
2. CLIENT(SCRITTURA CARATTERI) Socket -> OutputStream -> PrintWriter (println)
3. THREAD SERVER (LETTURA NUMERI) InputStream -> DataInputStream (readInt)
4. TRHEAD SERVER (LETTURA CARATTERI) InputStream -> InputStreamReader -> BufferedReader (readLine)

SERVER MANDA COSE AL CLIENT (dentro try)

1. CLIENT (LETTURA CARATTERI) Socket -> InputStream -> InputStreamReader -> BufferedReader (readLine)
2. CLIENT (LETTURA NUMERI) Socket -> InputStream -> DataInputStream
3. THREAD SERVER (SCRITTURA CARATTERI) OutputStream -> PrintWriter (println)
4. THREAD SERVER(SCRITTURA NUMERI) OutputStream -> DataOutputStream

CASO DI SINCRONIZZAZIONE (PIÙ THREAD MODIFICANO UN DATO)

1. Creo classe a parte per il dato
2. Metto variabili private, e moduli public synchronized
3. Istanzio un oggetto della classe nel server
4. Lo inserisco come parametro nel ThreadServer
5. Se usato, metto nel try classeDato.modulo (Ricordare costruttore)

FINE THREAD SERVER

try { ts.close(); } catch (IOException e){}